

GitHub Commit and DVC Setup

Here is the exact flow your collaborators should follow, step-by-step, no extras.



1. Get the project (first time only)

Clone the repo

```
</> Bash
git clone https://github.com/sandipan-ds/ai_grievance_system.git
cd ai_grievance_system
```



2. Get your working branch (your feature branch)

See available branches

```
</> Bash
git branch -a
```

Switch to your branch (example: `feature/sand`)

```
</> Bash
git checkout -b feature/sand origin/feature/sand
```



Now they have exact same files as your branch



3. Create their own branch (recommended)

```
</> Bash
git checkout -b feature/<their-name>
```

Example:

```
</> Bash
```

```
git checkout -b feature/rahu1
```



4. Make changes locally

Edit files normally.



5. Stage + commit changes

</> Bash

```
git add .  
git commit -m "Describe your changes"
```



6. Push their branch to GitHub

</> Bash

```
git push -u origin feature/<their-name>
```



7. Create Pull Request (PR)

On GitHub:

- Go to repo
- Click **Compare & Pull Request**
- Select:
 - base → `feature/sand` (or main, depending on flow)
 - compare → their branch



8. If your branch gets updated (important)

Before continuing work:

</> Bash

```
git checkout feature/sand
git pull origin feature/sand
```

Then update their branch:

```
</> Bash
git checkout feature/<their-name>
git merge feature/sand
```

Simple mental flow

1. Clone repo
2. Checkout base branch (`feature/sand`)
3. Create own branch
4. Work → commit → push
5. Open PR

Rules (important)

- Never work directly on `main`
- Never push directly to `feature/sand`
- Always create your own branch
- Always use PR for review

If you want, I can also give a **clean branching strategy** (like industry standard Git flow) tailored for your project.